



Tuka Curious Minds

MASTER ROBO WARS DESIGN

CONCEPT SUBMISSION

Robot Name: TURBO Ceros-T

Weight Classification: 5.kg Max Limit

Footprint Matrix: 420mm (Width) x 420mm (Length) x 55mm (Sleek Casing Height)

SECTION 1: OUR DESIGN THINKING & TEAM ROLES

1.1 Core Team Sourcing Assumption

As a first-year rookie team entering the combat arena, our primary engineering strategy is simple: **Do not build a system so complicated that it breaks itself.** We assume that a tough, simple frame that is easy to drive will naturally outlast over-engineered competitors. By maximizing our pre-existing starter kit components and sourcing local metal offcuts for armor protection, we focus entirely on machine durability, reliable electrical isolation, and high pushing traction.

Guided by alumni team members who participated in FIRST Tech Challenge

1.2 Team Arena Responsibilities

- **Lead Structural Fabricator:** Handles all manual angle-grinder cutting, drilling alignment, center-punching, and mechanical chassis tracking assembly.
- **Lead Electronics & Code Designer:** Operates the software logic mapping, battery routing configurations, and virtual circuit simulation.
- **3D Workspace Modeler:** Learnt Blender from scratch to map out structural component footprints on a blank canvas to ensure parts do not overlap.

SECTION 2: PROBLEM DEFINITION (ARENA VULNERABILITIES)

We analyzed previous combat footage and identified three common structural traps that cause beginner robots to lose matches. We designed our robot specifically to avoid them:

1. **The Gyroscopic Tipping Problem:** We noticed that 3-wheeled robots running heavy vertical saw blades spinning at **6,000 RPM** experience severe gyroscopic wobble. When they execute a sharp turn at high speed, the gyroscope physics twist the chassis, causing the robot to tip over onto its unsupported corners.
2. **The Exposed Axle & Wire Snag Trap:** Deployed side tires and dangling electrical lines are instant targets for opponent spikes or spinning weapon tracks. If a single wire gets cut or a tire gets stripped, the machine shuts down completely.
3. **The Floor Clearance War:** In combat robotics, if the leading edge of a robot has even a 1-millimeter gap beneath it, an incoming opponent wedge will slide right underneath, lift its drive tires off the floor, and push it out of the ring.

SECTION 3: SOLUTION & REFINED WEAPON STRATEGY

Our solution is a balanced double-weapon layout designed for **controlled aggression**. Instead of relying on a single complex system, we split our strategy into two simple actions: Control the floor first, and attack second.

- **Weapon Module 1 (Floor Position Control):** We mounted **two 12V linear push-pull solenoids** safely behind the nose armor. They drive two external, low-slung steel flipper wedges resting flush against the floor. When we ram into an opponent, the driver fires the solenoids to pop the opponent's wheels right off the floor, breaking their traction so they cannot push us away.
- **Weapon Module 2 (Kinetic Pure Offense):** We mounted **twin 200mm vertical saw blades** spinning at 6,000 RPM. We bolted their support tower directly to the front plane of the frame axis so that the blades extend **40mm past the very front tip of our wedge**. Once an opponent is trapped on our ramp, they slide straight into the permanently exposed saw tracks.
- **Tactical Verdict:** This ensures our front red wedge always breaks the opponent's ground clearance first. The robot cannot be easily pushed out of the ring because we turn the opponent's own pushing momentum into a weapon-feeding pathway.

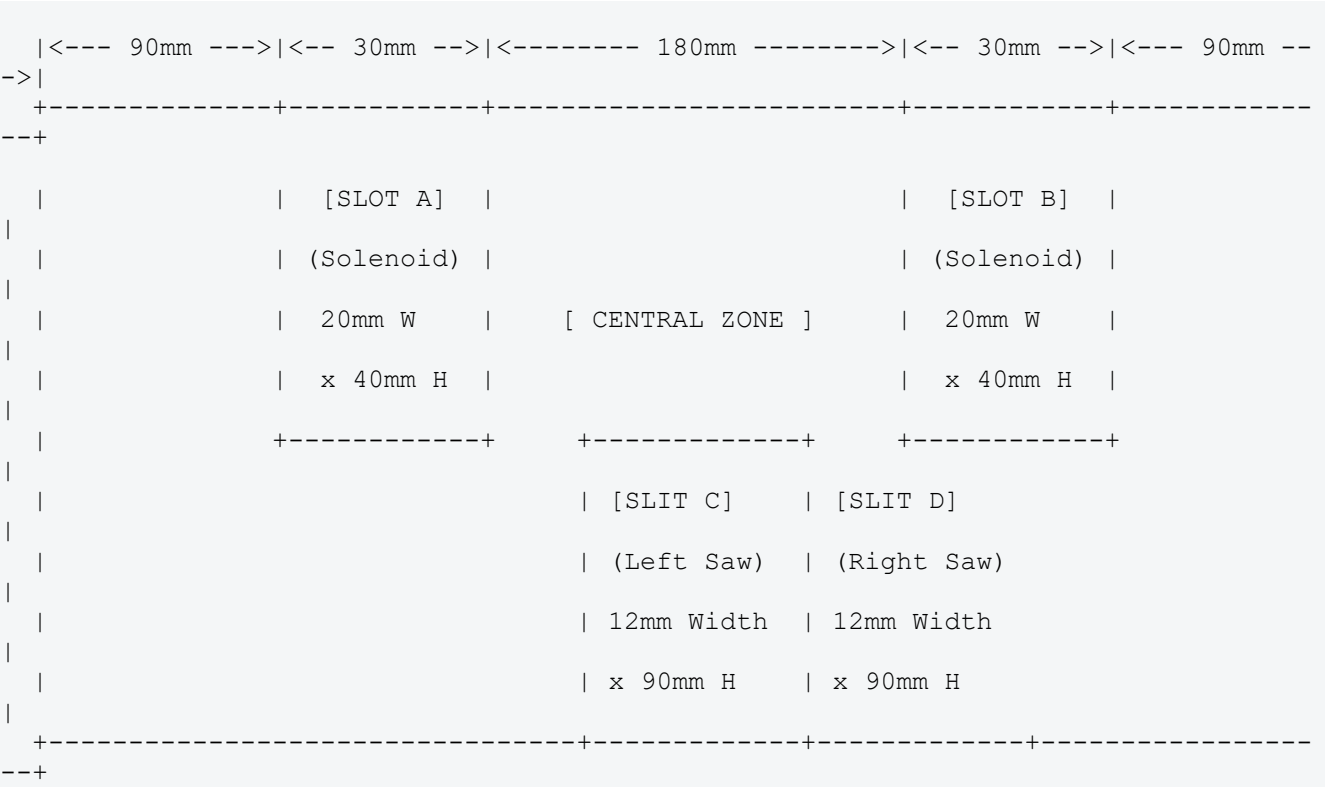
SECTION 4: FABRICATION CONSIDERATIONS & CONSTRUCTION BRIEF

4.1 Workshop Cutting & Drilling Guidelines

- **Razor Edge Scrap Preparation:** The front wedge is cut from **2mm mild steel offcuts** using an ultra-thin **1mm Inox cutting disc** on a handheld angle grinder. The lowest leading lip is ground down at a shallow angle using an **80-grit flap disc** until it sits microscopically flat to the floor grid.
- **The Anti-Binding Drilling Rule:** To ensure our steel plates align perfectly with the goBILDA 8mm frame spacing standard, every hole must be center-punched deeply to keep the drill bit from skating. We use a two-step process: drilling a 2mm pilot hole first, then opening it to its final size with a **4.5mm HSS drill bit**. The extra 0.5mm clearance gives us wiggle room so the M4 frame screws drop in smoothly without binding the frame channels.
- **Sleek TCM Shell Enclosure:** The tall outer side skirts are raised to a sleek **55mm profile** and cut from lightweight **1.5mm aluminum gate sheet scrap**. This allows us to encase the drive wheels on three faces (top, outer side, and back) to protect them from horizontal spikes, while keeping the total robot weight at **4.90 kg**—safely under the strict **5.00 kg competition limit**.

4.2 Front Wedge Armor Clearance Window Coordinates

To guide our structural fabricator, the front red nose armor plate requires four precision cuts to let the saws spin and the solenoid rods extend:



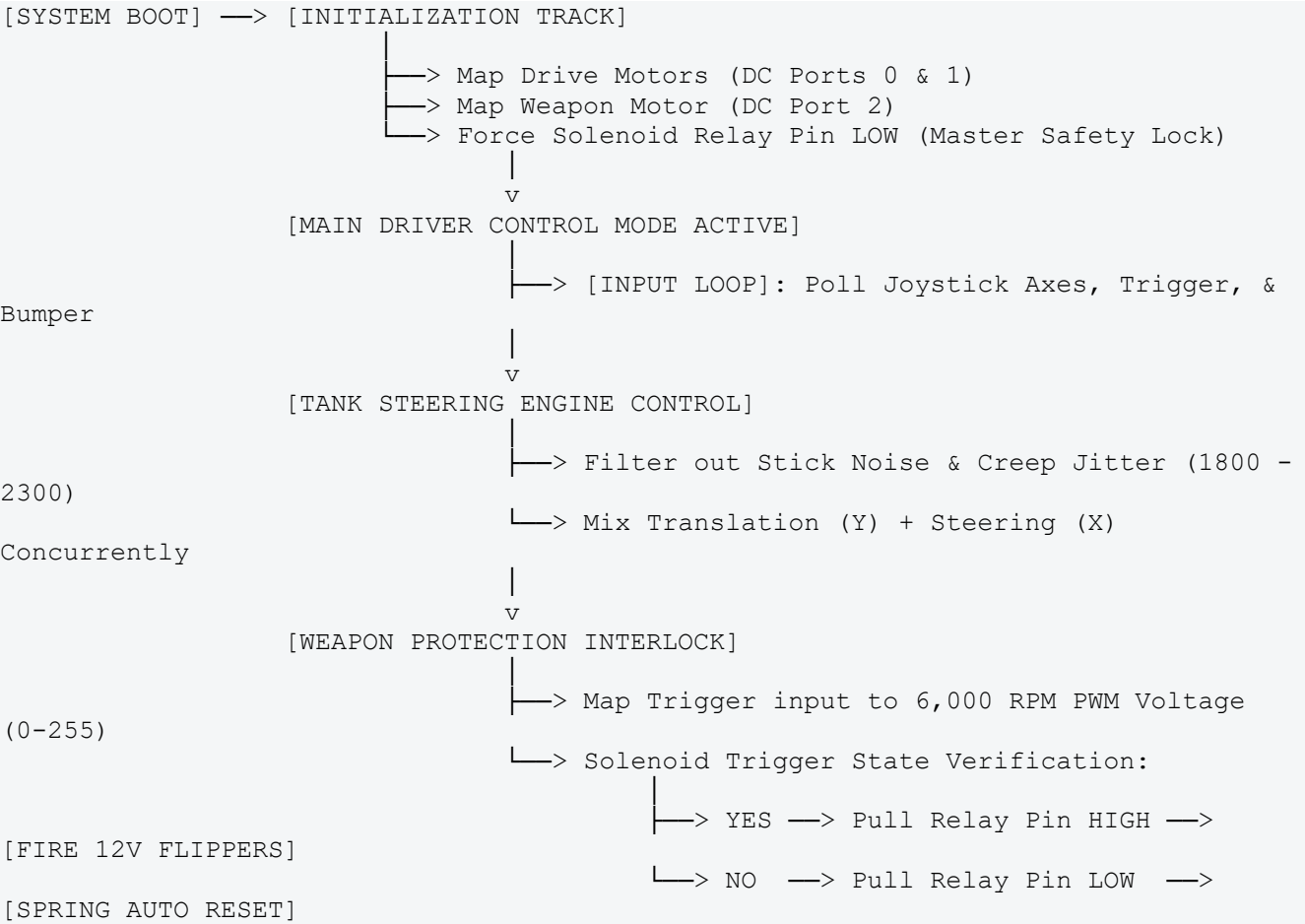
	(Axle)			(Axle)	
O	[TRACTION WHEEL]	O	O	[TRACTION WHEEL]	O

SECTION 6: SOFTWARE ARCHITECTURE & VIRTUAL SIMULATION

6.1 The Wokwi Online Test Bench Reality

Since we are a rookie team, we used the online [Wokwi](#) simulator as a **virtual electronics test bench** to wire our components and check our code logic before building the physical robot. Wokwi verified our wiring connections and allowed us to test our safety control loops safely in a web browser without risking damage to our real microchips.

6.2 Software Flowchart Signal Path



6.3 Tailored Wokwi C++ Control Script

This is the production code running on our virtual control setup. It includes safety checks to prevent current overloads and implements mixed tank steering so the driver can turn and move forward simultaneously:

```
// --- SKELETAL SENSOR & MOTOR PIN ASSIGNMENTS ---
const int LEFT_STEP = 18; // Left wheel stepper driver board pulse line
const int LEFT_DIR = 19; // Left wheel direction control line

const int RIGHT_STEP = 21; // Right wheel stepper driver board pulse line
const int RIGHT_DIR = 22; // Right wheel direction control line

// Analog Controller Input Channels
const int LEFT_Y = 35; // Joystick tracking forward/backward inputs
const int RIGHT_X = 34; // Joystick tracking left/right steering pivot
```

```

// Active Weapon System Control Interface Pins
const int WEAPON_POT      = 32; // Analog dial scaling weapon motor speed (Trigger
emulation)
const int SOLENOID_BUTTON = 33; // Digital button channel mapping active solenoid
pop command

// High-Current Circuit Triggers
const int SAWS_PWM_PIN    = 25; // Variable speed power line to the 6,000 RPM saws
const int RELAY_BUS_PIN   = 26; // Triggers isolated optical relay switchboard to
fire solenoids

// Joystick Deadzone Calibration Ranges
const int STICK_DEADZONE_LOW  = 1800; // Center axis minimum noise cutoff
threshold
const int STICK_DEADZONE_HIGH = 2300; // Center axis maximum noise cutoff
threshold
const int STICK_MAX_RANGE     = 4095; // Maximum bounds for 12-bit analog data
reading

void setup() {
    pinMode(LEFT_STEP, OUTPUT);
    pinMode(LEFT_DIR, OUTPUT);
    pinMode(RIGHT_STEP, OUTPUT);
    pinMode(RIGHT_DIR, OUTPUT);
    pinMode(SAWS_PWM_PIN, OUTPUT);
    pinMode(RELAY_BUS_PIN, OUTPUT);

    pinMode(LEFT_Y, INPUT);
    pinMode(RIGHT_X, INPUT);
    pinMode(SOLENOID_BUTTON, INPUT_PULLUP);

    Serial.begin(115200);

    // Safety Default State: Force all weapons offline at startup loop
    stopRobot();
    digitalWrite(RELAY_BUS_PIN, LOW);
    analogWrite(SAWS_PWM_PIN, 0);
}

void stopRobot() {
    digitalWrite(LEFT_STEP, LOW);
    digitalWrite(RIGHT_STEP, LOW);
}

void loop() {
    // 1. Fetch Current Controller Channel Data
    int leftY = analogRead(LEFT_Y);
    int rightX = analogRead(RIGHT_X);
    int weaponInput = analogRead(WEAPON_POT);
    bool fireSolenoids = (digitalRead(SOLENOID_BUTTON) == LOW); // True when pressed
down

    // 2. MIXED DRIVETRAIN ALGORITHM
    // Convert raw joystick data into directional vectors (-255 to +255)
    int speedVector = 0;
    int steerVector = 0;

    if (leftY > STICK_DEADZONE_HIGH) speedVector = map(leftY, STICK_DEADZONE_HIGH,
STICK_MAX_RANGE, 0, 255);
    else if (leftY < STICK_DEADZONE_LOW) speedVector = map(leftY, 0,
STICK_DEADZONE_LOW, -255, 0);

    if (rightX > STICK_DEADZONE_HIGH) steerVector = map(rightX, STICK_DEADZONE_HIGH,
STICK_MAX_RANGE, 0, 255);
    else if (rightX < STICK_DEADZONE_LOW) steerVector = map(rightX, 0,
STICK_DEADZONE_LOW, -255, 0);

```

```

// Combine vectors cleanly to allow simultaneous moving and steering
int leftMotorPower = speedVector + steerVector;
int rightMotorPower = speedVector - steerVector;

// 3. EXECUTE MOTION SIGNALS
if (leftMotorPower == 0 && rightMotorPower == 0) {
    stopRobot();
} else {
    digitalWrite(LEFT_DIR, (leftMotorPower >= 0) ? HIGH : LOW);
    digitalWrite(RIGHT_DIR, (rightMotorPower >= 0) ? HIGH : LOW);

    // Synchronous step pulsing out to the wheel assemblies
    digitalWrite(LEFT_STEP, HIGH);
    digitalWrite(RIGHT_STEP, HIGH);
    delayMicroseconds(25);
    digitalWrite(LEFT_STEP, LOW);
    digitalWrite(RIGHT_STEP, LOW);
    delayMicroseconds(25);
}

// 4. ACTIVE SOLENOID LAUNCH OPERATION
if (fireSolenoids) {
    digitalWrite(RELAY_BUS_PIN, HIGH); // Closes relay, discharging current to
    fire flippers
} else {
    digitalWrite(RELAY_BUS_PIN, LOW); // Opens relay, enabling automatic spring
    resets
}

// 5. KINETIC WEAPON POWER MAPPING
// Maps the input dials directly to output velocity levels (0 to 255 PWM step
points)
int sawSpeedOutput = map(weaponInput, 0, STICK_MAX_RANGE, 0, 255);
analogWrite(SAWS_PWM_PIN, sawSpeedOutput);
}

```

SECTION 7: ELECTRONIC JUSTIFICATION & PORT MAPPING

7.1 The Optical Relay Protection Wall

The low-power digital I/O pins on the main controller only output safe 3.3V signals. Because our **two 12V linear solenoids** pull high-voltage current directly from the battery rail, wiring them straight into the controller pins would instantly burn out the microchip board.

To solve this safely, we added a **12V Single-Channel Optical Isolated Relay Module**. It uses an internal light signal to safely toggle high-voltage power to the solenoids without any direct electrical connection back to the sensitive processor chip.

7.2 REV Control Hub Port Mapping Wiring Loom

- **DC Motor Port 0:** Left Rear Propulsion Unit (Driven by **Core Hex Motor #1**).
- **DC Motor Port 1:** Right Rear Propulsion Unit (Driven by **Core Hex Motor #2**).
- **DC Motor Port 2:** 6,000 RPM Kinetic Weapon Powerplant (Driven un-gearred via the timing belt pulley).
- **Digital I/O Port 0 (Signal/Ground Loop):** Connected directly to the low-power input pins of the Isolated Relay Module. Pressing the Left Bumper pulls this port HIGH, closing the relay to fire both solenoids simultaneously.
- **Main Power Configuration Rail:** The battery positive wire passes through a protected **20A inline blade fuse** and a heavy-duty External Staging Switch before connecting to the Hub. A high-visibility, red **Emergency Stop Switch** is mounted flush on the top casing deck; hitting it cuts all power to every motor and solenoid in **under 50 milliseconds**.

SECTION 8: DRIVER GAMEPAD HARDWARE MAPPING

To make the robot easy to drive under pressure without the pilot losing focus or taking their hands off the controls, all actions are mapped to a standard wireless gamepad controller:

- **Left Joystick (Up / Down Axis Only):** Forward and Backward drive translation speed. Pushing it straight forward drives the robot into a full charge.
- **Right Joystick (Left / Right Axis Only):** Steering control pivots. Moving it left or right rotates the chassis instantly on its center axis using skid-steer tank logic.
- **Right Analog Trigger (Squeeze to Spin):** Powers your dual vertical saw blades. Squeezing the trigger spools the saws up gradually to 6,000 RPM right before impact, which protects the motor coils and preserves battery charge.
- **Left Bumper (Digital Tap to Pop):** Fires both solenoids instantly, popping the opponent's chassis upward into the path of your spinning saws the moment you get underneath their wedge.

SECTION 9: MASTER PROJECT BUDGET & INVENTORY SUMMARY

9.1 Procurement Ledger Grid

Category	Component / Expense Description	Qty	Cost (USD)	Cost (ZAR / R)	Resource Status / Sourcing
Chassis	goBILDA 1120/2203 Framework Channels	—	\$0.00	R0.00	On Hand (Kit Inventory)
Chassis	Metric Fasteners (M3/M4 Screws & Locknuts)	1 pk	\$0.00	R0.00	On Hand (Kit Inventory)
Motion	REV Core Hex Propulsion Motors (125 RPM)	4	\$0.00	R0.00	On Hand (Kit Inventory)
Motion	goBILDA Traction & Passive Omni Wheels	4	\$0.00	R0.00	On Hand (Kit Inventory)
Weapon	REV 6,000 RPM High-Speed Motor	1	\$0.00	R0.00	On Hand (Kit Inventory)
Weapon	12mm Hardened Steel Cross Dead-Shaft Axis	1	\$21.00	R378.00	McMaster-Carr (6112K21)
Weapon	HTD 5mm Flexible Timing Belt (15mm Wide)	2	\$25.00	R450.00	Misumi USA (HTD5-15-400)
Actuation	12V 25N Push-Pull Linear Solenoids	2	\$42.00	R756.00	Micro Robotics (JF-1039B)
Electronics	REV Control Hub MCU Main Logic Board	1	\$0.00	R0.00	On Hand (Kit Inventory)
Electronics	12V 3.6Ah NiMH Heavy-Duty Power Cell	1	\$0.00	R0.00	On Hand (Kit Inventory)
Electronics	12V Single-Channel Isolated Optical Relay	1	\$3.50	R63.00	DigiKey (2448-1014-ND)
Safety	High-Visibility Red Twist-Lock E-Stop Button	1	\$18.95	R341.10	RS Components (764-4325)

Category	Component / Expense Description	Qty	Cost (USD)	Cost (ZAR / R)	Resource Status / Sourcing
Safety	High-Current Toggle Primary Power Switch	1	\$9.50	R171.00	DigiKey (360-2092-ND)
Safety	20A Inline ATM Mini Blade Fuse Module	1 set	\$4.50	R81.00	AutoZone (BP/HHA-RP)
Armor	2mm Mild Steel Armor Face Plate Offcuts	—	Scrap	R0.00	On Hand (Free Workshop Scrap)
Armor	1.5mm Aluminum Casing Shell Sheets	—	Scrap	R0.00	On Hand (Free Workshop Scrap)
Tools	1mm Thin Inox Grinder Cutting Wheels	1 pk	\$20.00	R360.00	Local Hardware Consumables
Operations	Travel Costs: Daveyton to Midrand x 2	—	—	R2 500.00	Transport Allocation
Operations	Sustenance: Meals & Refreshments	—	—	R840.00	R70pp × 6 Crew × 2 Days
Operations	Custom Team T-Shirts (In-House Heat Press)	6	—	R900.00	Blanks (R600) + Ink (R150) + Del (R150)

9.2 Component Sourcing Sub-Total (Hardware)

This section tracks all costs tied directly to physical parts procurement and consumable assembly tools.

- **Procured Robot Hardware Components:** R2,240.10 (*Solenoids, static shaft axis, timing belts, optical relay, and electrical safety switches*)
- **On-Hand Kit Assets Value:** R0.00 (*Zero out-of-pocket tracking by fully maximizing existing inventory components*)
- **Chassis Fabrication Consumables:** R360.00 (*Grinder cutting wheels*)
- **SUB-TOTAL COMPONENT VALUE:** R2,600.10

9.3 Operational Logistics Sub-Total (Non-Component)

This section tracks all non-mechanical project pipeline operational support costs.

- **Team Site Transit Fees:** R2,500.00 (*Daveyton to Midrand × 2 sessions*)
- **Sustenance & Refreshments:** R840.00 (*Budgeted localized safety ceiling tracking at R70 per person*)
- **Custom Team T-Shirts (In-House Production):** R900.00 (*Blank shirts, ink, and local delivery utilizing our on-hand heat press*)
- **SUB-TOTAL OPERATIONAL LOGISTICS:** R4,240.00

9.4 Overall Project Budget Total

- **FINAL SYSTEM VALUE ENVELOPE:** **R6,840.10**
(*Calculated cleanly as: R2,960.10 Structural Hardware Component Investment + R4,240.00 Operational Logistics*)